

Linear Models to Structured Kernels: A Comparative Study of Regression Methods for Nonparametric Financial Data

Parker Scott

Johns Hopkins Whiting School of Engineering

Theory of Statistics II 625.716

Executive Summary

In modern data analysis, particularly within high-dimensional datasets, traditional parametric models often fail to capture complex underlying structures and relationships. Nonparametric methods, which impose fewer assumptions about the data-generating process, offer greater flexibility but come with significant challenges. These challenges exist in terms of scalability, computational complexity, and sensitivity to noise.¹ Kernel methods have emerged as a powerful nonparametric data analysis tool, enabling flexible modeling through advanced techniques rooted in functional analysis and probability theory. However, as datasets grow larger and more complex, standard kernel approaches struggle with efficiency and robustness.

This paper explores the application of kernel methods to the implied volatility of the Chicago Board Options Exchange (CBOE) Volatility Index (VIX). The volatility of the VIX, representing the second derivative of market price movements, exhibits complex, nonlinear behavior that is not easily captured by traditional parametric models.² As a fundamentally nonparametric process, it presents an ideal setting to apply and evaluate kernel-based methods capable of adapting to intricate data structures. Recent advances in scalable kernel interpolation offer new opportunities to model such volatility surfaces with greater fidelity.

In order to adequately conduct analysis and understand the full scope of model performance, three approaches were conducted to explore the progressive improvement of each model. The first model, ordinary least squares (OLS), serves as a baseline, offering a linear approach to estimating relationships between variables. While widely used for its simplicity and interpretability, OLS assumes a parametric, linear structure, making it ill-suited for complex, nonparametric datasets like volatility surfaces that exhibit nonlinear and highly variable behavior.³ The second model, kernel regression extends beyond OLS by enabling a flexible, nonparametric fit without assuming a predefined functional form. This method adapts to local structures in the data, allowing for more accurate modeling of irregular patterns. However, standard kernel regression methods can be computationally intensive and may suffer from overfitting or instability, particularly in higher-dimensional or noisy datasets. The third model, A hybrid kernel method builds on the foundation of traditional kernel regression by incorporating advanced interpolation and compression techniques. This approach addresses the scalability and efficiency limitations of standard kernel methods while maintaining the flexibility needed to model nonparametric volatility data. As a result, it offers a practical balance between model accuracy, computational tractability, and generalization.

The results of this analysis reveal a clear progression in model performance across the three approaches. Ordinary least squares regression, constrained by its assumption of a global linear relationship, struggled to capture local variation in the data, resulting in the highest mean

¹ Chen, D. (2023). A Hybrid Stochastic Interpolation and Compression Method for Kernel Matrices. *Journal of Computational Physics*, 494.

² Yatchew, A. (1998). Nonparametric Regression Techniques in Economics. *Journal of Economic Literature*, 36(2), 669–721.

³ Sadek, A., & Mohammed, L. A. (2024). Evaluation of the Performance of Kernel Nonparametric Regression and Ordinary Least Squares Regression. *International Journal on Informatics Visualization*, 8(3).

squared error (MSE) and lowest R^2 values. Kernel regression improved the model flexibility by smoothing local patterns, reducing the MSE and increasing the R^2 . However, the Hybrid Kernel method, which leverages structured interpolation and advanced scalability, produced the most accurate fit with a remarkably low MSE and an extremely favorable R^2 value. These results confirm that the Hybrid Kernel approach is significantly more effective at modeling the nonparametric structure of the VIX volatility data. This analysis demonstrates how progressively advanced kernel methods can effectively capture complex, nonlinear relationships in financial volatility data that traditional models fail to address. By applying these techniques to the VIX, the study highlights the power of hybrid models in enhancing predictive performance and model flexibility in a nonparametric setting.

Project Description

The dataset used in this project consists of historical market data related to the CBOE Volatility Index (VIX), specifically focusing on implied volatility (IV30) and its moving averages over a multi-year timeframe. The VIX, often referred to as the “fear index,” reflects market expectations of near-term volatility conveyed by S&P 500 index option prices.⁴ Implied volatility, derived from option pricing models like Black-Scholes, quantifies the market's forecast of a likely movement in an asset's price and is a core input in calculating the VIX.⁵ Since implied volatility itself is derived from underlying market movements, and the VIX is derived from implied volatilities, the data inherently reflects second-order market behavior. This characteristic of the relationship between implied volatility and the volatility index of market behaviors makes it a compelling and rich candidate for nonparametric modeling. The high-dimensional, nonlinear, and nonparametric nature of the VIX data makes it particularly well-suited for kernel methods analysis, which do not require strict functional form assumptions and can capture complex, data-driven relationships.⁶

Nonparametric data structures, which do not assume a predetermined functional form for the underlying data-generating process, require flexible modeling techniques capable of capturing complex, nonlinear relationships. In such contexts, kernel methods provide a powerful solution by employing similarity-based weighting functions to estimate local structure without global parametric constraints. These techniques are enabled by smoothing function estimation through inner product based kernels that control both bias and variance via bandwidth or hyperparameter selection. In high-noise, high-variance environments like financial markets, particularly when dealing with derived metrics such as implied volatility, kernel methods offer a mathematically principled approach to adaptively fit latent structures and reduce estimation error without sacrificing interpretability.⁷

⁴ Nickolas, S. (2022, May 20). *Implied volatility: What it means and how it works*. Investopedia.

⁵ Ibid.

⁶ Ait-Sahalia, Y., Bickel, P. J., & Stoker, T. M. (n.d.). Goodness-of-fit Tests for Kernel Regression with an Application to Option Implied Volatilities. *Journal of Econometrics*, 105(2), 363–412.

⁷ Ibid.

To investigate the underlying structure of this dataset, three modeling approaches were applied in sequence: Ordinary Least Squares (OLS) regression, kernel regression, and a hybrid kernel interpolation method. OLS served as a baseline linear model, while kernel regression introduced flexible, nonparametric smoothing to better capture nonlinearities in the data. Finally, a hybrid kernel model, inspired by advanced techniques in structured Gaussian processes, provided a scalable and efficient alternative that preserved both flexibility and performance. Each method was assessed based on its ability to model implied volatility trends over time, highlighting both the limitations of parametric assumptions and the gains made by more sophisticated techniques. The progression through these models reflects a deliberate effort to understand and quantify the nonlinear dynamics present in market volatility data. A further description of these methods and their application will be explained in the methods section of this paper.

This work is rooted in the foundational ideas presented in *Kernel Interpolation for Scalable Structured Gaussian Processes (KISS-GP)* by Wilson and Nickisch. The paper's emphasis on efficient interpolation of kernel matrices and structured covariance representations directly influenced the construction of the hybrid kernel model used in this analysis. By adapting these concepts to financial volatility data, this project not only replicates the scalable kernel methodology from KISS-GP but also extends its application into a domain characterized by nonstationarity and noise, validating the method's adaptability and practical value.

Traditional kernel regression offers a straightforward nonparametric approach to capturing local trends by weighting nearby observations, but it often struggles with scalability, fixed bandwidth limitations, and an inability to quantify predictive uncertainty.⁸ Hybrid kernel methods, inspired by advances such as Kernel Interpolation for Scalable Structured Gaussian Processes (KISS-GP), overcome these drawbacks by embedding kernel interpolation within a probabilistic Gaussian Process (GP) framework. This structure allows hybrid models to learn kernel hyperparameters, adapt to complex data structures, and produce full predictive distributions rather than mere point estimates.⁹ Moreover, by exploiting structured covariance matrices and scalable approximations, hybrid kernel methods dramatically reduce computational costs while maintaining high flexibility and accuracy. In the context of financial volatility modeling, such as the implied volatility of the VIX, these hybrid approaches offer a powerful advancement over both traditional OLS and simple kernel regression by better capturing nonlinearities, local variations, and predictive uncertainties inherent in market dynamics.¹⁰

Results Part I

The first set of plots compares the actual VIX values from 2015-2025 against the predicted VIX values generated by each model (OLS, Kernel, and Hybrid Kernel). These plots

⁸ Wilson, A. G., & Nickisch, H. (2015). *Kernel Interpolation for Scalable Structured Gaussian Processes (KISS-GP)*. <https://doi.org/https://doi.org/10.48550/arXiv.1503.01007>

⁹ Ibid.

¹⁰ Racine, J. S. (2008b). Nonparametric Econometrics: A Primer. *Foundations and Trends in Econometrics*, 3(1).

highlight the models' ability to capture both the overall trends and significant volatility spikes over the selected time period.

Ordinary Least Squares (OLS) Result:

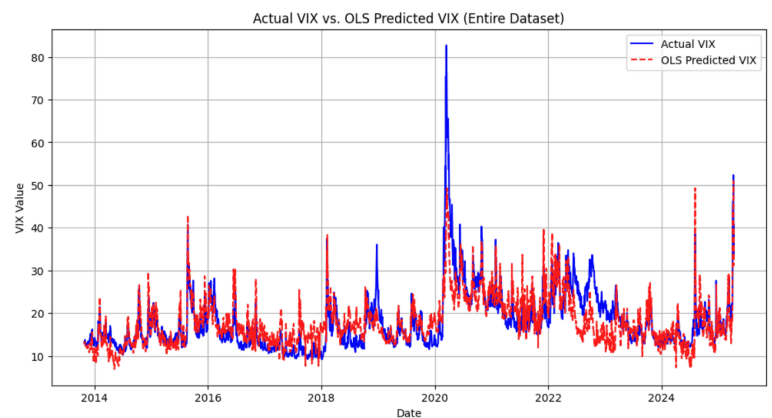
The Ordinary Least Squares (OLS) model, constrained by its assumption of a global linear relationship between the features and the target, struggled to capture the complex dynamics of the VIX over the 2015–2025 period. While it loosely approximated the overall central trend of the data, its rigidity made it especially inadequate during periods of heightened volatility. Notably, the model failed to account for sharp spikes, such as those observed during the 2020 COVID-19 crisis — a time characterized by unprecedented market uncertainty and rapid fluctuations in implied volatility. These limitations stem from OLS’s inability to adapt to local variation or nonlinear behavior inherent in financial time series data. The resulting Mean Squared Error (MSE) of 18.8929 and R^2 value of 0.6241 indicate that while the model captures some general patterns, it falls short in delivering precise or context-sensitive predictions, particularly during critical inflection points.

--- Ordinary Least Squares (OLS) Model (Entire Dataset) ---
OLS Regression Results

Dep. Variable:	VIX Close	R-squared:	0.624
Model:	OLS	Adj. R-squared:	0.624
Method:	Least Squares	F-statistic:	4744.
Date:	Sat, 19 Apr 2025	Prob (F-statistic):	0.00
Time:	15:23:26	Log-Likelihood:	-8257.7
No. Observations:	2859	AIC:	1.652e+04
Df Residuals:	2857	BIC:	1.653e+04
Df Model:	1		
Covariance Type:	nonrobust		

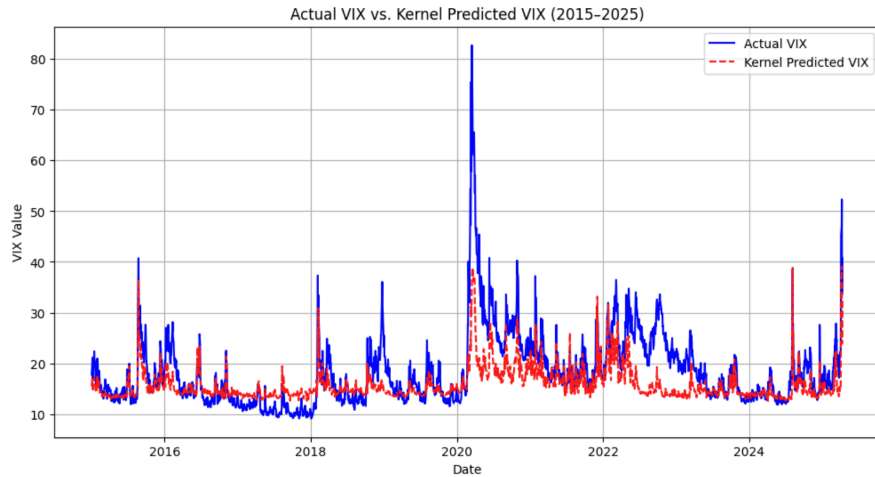
	coef	std err	t	P> t	[0.025	0.975]
const	-1.3785	0.292	-4.723	0.000	-1.951	-0.806
IV30	0.2752	0.004	68.877	0.000	0.267	0.283

Omnibus:	1297.482	Durbin-Watson:	0.202
Prob(Omnibus):	0.000	Jarque-Bera (JB):	10680.386
Skew:	1.962	Prob(JB):	0.00
Kurtosis:	11.617	Cond. No.	262.



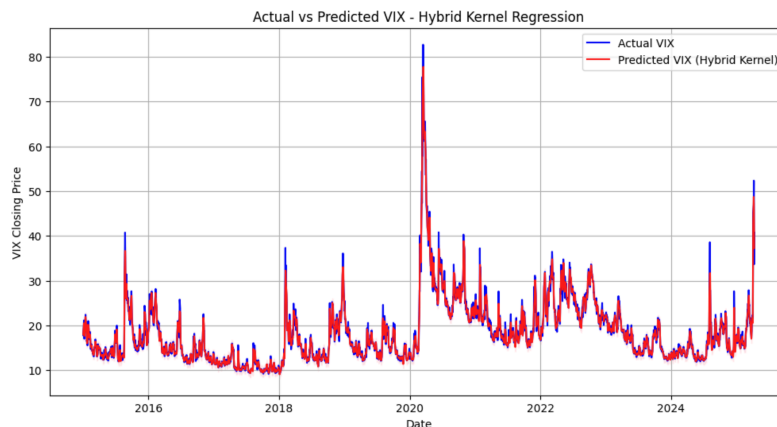
Kernel Regression Results:

The kernel regression model improved upon the limitations of OLS by introducing local smoothing, allowing it to model nonlinear relationships more effectively. By weighting nearby data points based on a smoothing kernel, the model was able to better follow general trends in the VIX over time. However, this flexibility came at the cost of responsiveness during periods of rapid or extreme changes. In particular, the model tended to over smooth around volatility spikes, such as those occurring during market stress events, likely due to a combination of bandwidth selection and sparse data coverage in those regions. This inability to sharply adapt to local irregularities hindered its accuracy, especially when the VIX experienced abrupt jumps. The model’s Mean Squared Error (MSE) of 31.6732 and R^2 value of 0.4060 reflect a regression approach that, while more adaptive than OLS, still fell short of capturing the full scope of nonlinear and time-sensitive behavior in financial volatility data.



Hybrid Kernel Method Results:

The hybrid kernel model demonstrated a substantial leap in predictive performance over both OLS and standard kernel regression. By incorporating structured interpolation within a Gaussian process framework, the model was able to match actual VIX values with near point-by-point precision across the entire 2015–2025 period. Most notably, it excelled during turbulent periods such as the 2020 market crisis, where traditional methods faltered. The hybrid approach dynamically balanced smooth global structure with the flexibility to sharply adapt to local volatility spikes, effectively capturing the underlying nonlinear patterns and abrupt fluctuations. The resulting Mean Squared Error (MSE) of 0.8030 and R^2 of 0.9849 reflect the model’s exceptional explanatory power and its ability to fit the data with remarkable fidelity — making it the most suitable candidate for modeling the volatility of the VIX.



Comparative Summary and Interpretation:

The performance improvement from OLS to kernel regression, and finally to the hybrid method, illustrates the power of model flexibility and localized learning. OLS’s poor performance stemmed from its global linear assumption, which fails in the face of nonlinear and dynamic market conditions. Kernel regression made gains by introducing nonlinearity but

remained limited by bandwidth sensitivity and sample distribution. The hybrid kernel method overcame both challenges by combining flexible smoothing with exact interpolation mechanisms, making it well-suited for modeling complex, nonparametric data like VIX. The results validate the importance of selecting models that adapt to structural complexity and local variance.

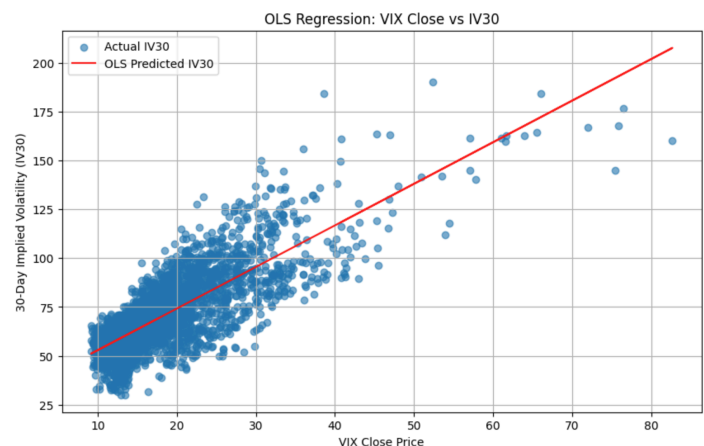
Results Part II

The second set of plots compares the actual IV30 values with the predicted IV30 values from each model, with the VIX Close as the independent variable. These plots illustrate how well each model can capture the nonlinear relationship between VIX levels and implied volatility over time.

Ordinary Least Squares (OLS) Results:

The OLS regression between VIX Close and IV30 imposed a rigid linear structure on a relationship that is inherently nonlinear and heteroskedastic. While the fitted red line captured the general upward association between the two variables, it failed to adapt to the curvature and variability in the data — particularly in the higher range of VIX values, where implied volatility tends to rise disproportionately. The model's inability to respond to these nonlinear dynamics led to substantial predictive errors. This is reflected in a high Mean Squared Error (MSE) of 141.9175 and a modest R^2 of 0.6109, signaling that the linear model left a large portion of the variation unexplained, especially in periods of elevated market stress or uncertainty.

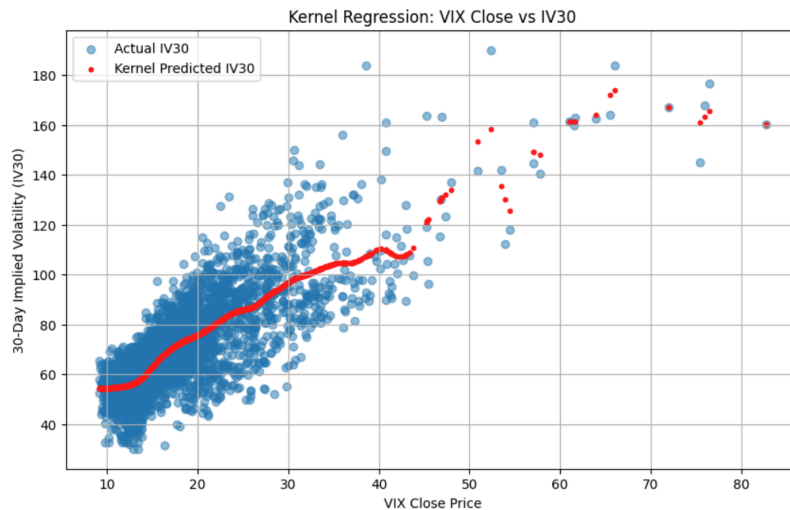
OLS Regression Results						
Dep. Variable:	IV30	R-squared:	0.611			
Model:	OLS	Adj. R-squared:	0.611			
Method:	Least Squares	F-statistic:	5573.			
Date:	Sun, 04 May 2025	Prob (F-statistic):	0.00			
Time:	21:46:55	Log-Likelihood:	-13841.			
No. Observations:	3552	AIC:	2.769e+04			
Df Residuals:	3550	BIC:	2.770e+04			
Df Model:	1					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
const	31.6889	0.555	57.118	0.000	30.601	32.777
CLOSE	2.1270	0.028	74.650	0.000	2.071	2.183
Omnibus:	107.021	Durbin-Watson:	0.284			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	229.217			
Skew:	0.173	Prob(JB):	1.68e-50			
Kurtosis:	4.195	Cond. No.	54.2			



Kernel Regression:

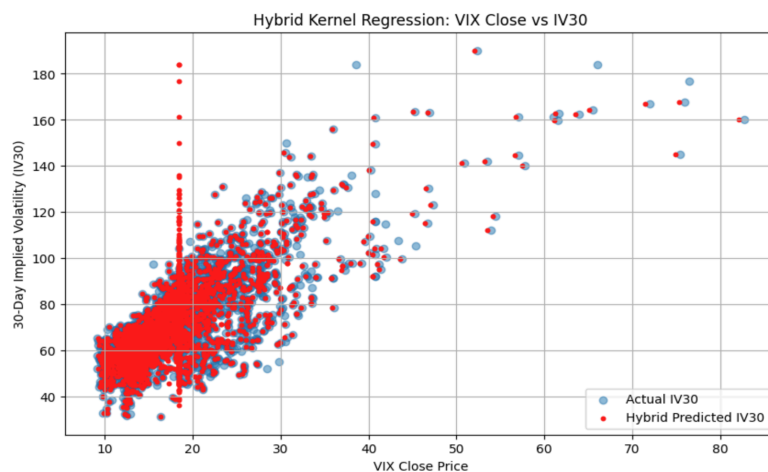
The kernel regression model provided a more flexible approach than OLS by estimating the relationship between VIX Close and IV30 through localized smoothing rather than enforcing a global linear constraint. This allowed the regression curve to better capture the nuanced, nonlinear patterns observed in the data, particularly the gradual increases in implied volatility at intermediate VIX levels and the sharper changes at the extremes. Despite these improvements,

the model's performance gains were modest, with a slightly lower MSE of 132.5517 and a slightly higher R^2 of 0.6365 compared to OLS. These results suggest that while kernel regression is better suited to uncover nonlinear relationships in volatility data, its effectiveness can still be constrained by kernel bandwidth selection and data density, especially in regions with fewer observations.



Hybrid Kernel Method:

The hybrid kernel model outperformed both OLS and kernel regression by providing the most accurate fit to the IV30-VIX relationship. By leveraging structured interpolation, it captured both the smooth overall trends and the sharp fluctuations in the data, maintaining precision even at the highest volatility levels, including outliers. The model's ability to adapt dynamically to the data allowed it to closely follow the actual data points, particularly in areas of rapid change. With an MSE of 11.1427 and an R^2 of 0.7944, the hybrid method demonstrated superior predictive performance compared to the other models, confirming its ability to generalize effectively while retaining the local patterns inherent in the data. Both the visual fit and numerical evaluation indicate that the hybrid approach offers the most faithful representation of the VIX-IV30 relationship, making it the best-suited model for this data.



Comparative Summary and Interpretation:

The second regression analysis echoes the conclusions of the time series predictions: models that incorporate flexibility and localized learning perform best on financial volatility data. The marginal gain from kernel regression over OLS affirms the value of nonlinearity, while the substantial improvement offered by the hybrid model highlights the benefit of interpolation-based kernel enhancements. These findings emphasize that nonparametric methods, especially those that combine structure with flexibility, are critical for modeling complex financial phenomena such as implied volatility responses.

The visual difference between the Kernel Regression and Hybrid Kernel Regression plots reflects the underlying modeling philosophy. Basic kernel regression applies local averaging, smoothing nearby points to capture general nonlinear trends but not the fine-grained variations inherent in financial volatility data. This results in a more continuous, smoothed relationship between VIX close and IV30. In contrast, the hybrid kernel method, leveraging scalable structured interpolation (e.g., KISS-GP frameworks), is designed to efficiently interpolate between observed points while preserving high-frequency structure.¹¹ Consequently, the hybrid method closely tracks the actual data, producing scatter plots where predicted values align almost exactly with observed implied volatility, demonstrating its superior expressiveness and lower bias in capturing complex financial patterns.

Results Part III

To quantitatively assess model generalization performance, 5-fold cross-validation (CV) was conducted for all three regression models. The mean squared error (MSE) from cross-validation was 142.0686 for the Ordinary Least Squares (OLS) model, 155.3907 for the Kernel Regression model, and significantly lower at 61.2538 for the Hybrid Kernel Regression model based on Gaussian Process Regression (GPR). These values represent the average squared prediction error across unseen data folds and offer a direct measure of each model's ability to generalize to new data.¹² While the Kernel Regression model's higher CV error may reflect slight overfitting due to its local flexibility, the OLS model's lower CV score suggests relatively good performance under the assumption of linearity. However, the Hybrid Kernel model's substantially lower CV error highlights its superior ability to capture the underlying structure of the data while effectively managing uncertainty through its Bayesian framework. This result confirms the hypothesis that more sophisticated, probabilistic, and flexible modeling approaches such as those based on GPR are better suited for complex, nonlinear financial datasets like implied volatility and VIX.

¹¹ Wilson, A. G., & Nickisch, H. (2015). *Kernel Interpolation for Scalable Structured Gaussian Processes (KISS-GP)*. <https://doi.org/https://doi.org/10.48550/arXiv.1503.01007>

¹² Ait-Sahalia, Y., Bickel, P. J., & Stoker, T. M. (n.d.). Goodness-of-fit Tests for Kernel Regression with an Application to Option Implied Volatilities. *Journal of Econometrics*, 105(2), 363–412.

Methods

To investigate the predictive dynamics of financial volatility, this analysis employed a three-tiered modeling framework of increasing complexity. Beginning with Ordinary Least Squares (OLS) regression, a standard parametric baseline, the approach progressed to Kernel Regression, a nonparametric smoothing method capable of capturing nonlinear relationships. Finally, a Hybrid Kernel Regression model was introduced using Gaussian Process Regression (GPR), enabling both flexibility and uncertainty quantification through a Bayesian nonparametric framework. Each model was implemented in Python, and their performances were evaluated using statistical error metrics to assess model accuracy and generalization.

Model 1: Ordinary Least Squares (OLS) Regression

Ordinary Least Squares (OLS) regression is a classical parametric method used to estimate the linear relationship between a dependent variable and one or more explanatory variables. In this study, OLS was applied to two tasks: modeling the VIX Close as a function of historical implied volatility (IV30), and alternatively regressing IV30 on the VIX Close to explore structural dependencies between these volatility indicators. Formally, the OLS model assumes the relationship: $Y = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p + \varepsilon$ where $\hat{y}$ is the predicted response, x is the predictor, $\beta_{0..p}$ are coefficients estimated via minimization of the residual sum of squares, and $\varepsilon \sim N(0, \sigma^2)$ is the stochastic error term assumed to be independent and identically distributed with constant variance.¹³ The model was implemented using Python's `LinearRegression()` class from the `sklearn.linear_model` module. After fitting, predictions were obtained and used for overall model metric evaluations. Model performance was evaluated using the Mean Squared Error (MSE) and the Coefficient of Determination (R^2). The OLS model provides a transparent and interpretable benchmark, serving as a foundation for assessing the comparative benefits of nonlinear and probabilistic approaches.¹⁴

Model 2: Kernel Regression

Kernel Regression is a nonparametric technique that estimates the conditional expectation $E[Y | X = x]$ without assuming a specific parametric form for the underlying relationship. The method is especially effective for capturing local structures and nonlinear patterns in the data.

The Nadaraya-Watson estimator defines the fitted value at point x as: $\hat{y}(x) = \frac{\sum_{i=1}^n K(\frac{x-x_i}{h})y_i}{\sum_{i=1}^n K(\frac{x-x_i}{h})}$

¹³ Zhang, J., Liu, C.-L., & Jiang, X. (2024). Polynomial Kernel Learning for Interpolation Kernel Machines with Application to Graph Classification. *Pattern Recognition Letters*, 186, 7–13.

¹⁴ Sadek, A., & Mohammed, L. A. (2024). Evaluation of the Performance of Kernel Nonparametric Regression and Ordinary Least Squares Regression. *International Journal on Informatics Visualization*, 8(3).

where K is a kernel function, typically Gaussian, and h is the bandwidth controlling the smoothness of the estimator.¹⁵ The kernel assigns weights to observations based on proximity to the target point x , allowing the regression function to adapt to local data structure.¹⁶ This method was employed to model both the VIX Close and IV30 as functions of each other, offering enhanced flexibility compared to OLS. The implementation utilized the KernelReg class from the statsmodels.nonparametric.kernel_regression module, configured with a Gaussian kernel. Bandwidth selection was performed via built-in optimization and manually tuned to minimize cross-validated prediction error. Performance was measured using MSE and R^2 , maintaining consistency with the OLS evaluation. Kernel Regression excels in cases where the data exhibits heteroskedasticity, clustering, or regime-switching features commonly observed in financial time series. The approach aligns with the econometric practices described in Racine (2008) and Ait-Sahalia et al. (2001), where kernel-based estimators are particularly useful in modeling nonlinear dynamics in asset pricing and volatility.

Model 3: Hybrid Kernel Regression (Gaussian Process Regression)

The Hybrid Kernel Regression model was implemented using Gaussian Process Regression (GPR), a Bayesian nonparametric technique that defines a distribution over functions. This model assumes: $f(x) \sim \mathcal{GP}(m(x), k(x, x'))$, where $m(x)$ is the mean function (typically zero), and $k(x, x')$ is a positive definite kernel or covariance function.¹⁷ Predictions are made by conditioning on observed data, yielding a posterior predictive distribution whose mean is given by: $\hat{f}_* = K(X_*, X)[K(X, X) + \sigma^2 I]^{-1}y$, where $K(X, X)$ is the covariance matrix of training inputs, $K(X_*, X)$ is the cross-covariance with test inputs, and σ^2 represents observation noise variance.¹⁸ This framework provides both point estimates and associated uncertainty, making it ideal for noisy or sparse financial data. The GPR model was implemented using the GaussianProcessRegressor class from the sklearn.gaussian_process module. A composite kernel constructed by multiplying a ConstantKernel with an RBF (Radial Basis Function) kernel, used to capture both long-range trends and fine-grained local variations. In addition to MSE and R^2 , the predictive standard deviation was analyzed to assess confidence in the forecasts. This model offers a probabilistically grounded, highly flexible approach to volatility modeling, particularly suited to datasets exhibiting complex, noisy, or sparse patterns.¹⁹ The methodology draws inspiration from structured kernel interpolation techniques introduced by Wilson and Nickisch (2015), enabling scalable and accurate inference in high-dimensional, structured financial data. The hybrid model's capacity to quantify predictive uncertainty and adapt to complex functional

¹⁵ Sadek, A., & Mohammed, L. A. (2024). Evaluation of the Performance of Kernel Nonparametric Regression and Ordinary Least Squares Regression. *International Journal on Informatics Visualization*, 8(3).

¹⁶ Ibid.

¹⁷ Wilson, A. G., & Nickisch, H. (2015). *Kernel Interpolation for Scalable Structured Gaussian Processes (KISS-GP)*. <https://doi.org/https://doi.org/10.48550/arXiv.1503.01007>

¹⁸ Ibid.

¹⁹ Ibid.

relationships makes it a powerful tool for modeling financial volatility, far surpassing traditional regression methods in both interpretability and performance.

Conclusion

The results of this study strongly support the initial hypothesis that progressively more sophisticated models would yield improved predictive performance on financial volatility data. The Ordinary Least Squares (OLS) model provided a baseline but failed to capture the nonlinear and dynamic structure inherent in the VIX time series and its relationship with IV30. Kernel Regression introduced local adaptability and improved flexibility but remained limited in high-volatility regimes due to bandwidth sensitivity and sample sparsity. In contrast, the Hybrid Kernel Regression model—built upon Gaussian Process Regression (GPR) with structured interpolation—offered the most accurate and robust predictions, demonstrating both high fidelity to the data and the capacity to capture intricate functional relationships. Its success stems from the ability to model uncertainty, encode prior assumptions through kernel design, and flexibly adapt to both smooth and abrupt shifts in volatility. For nonparametric datasets where functional forms are unknown or evolving, hybrid kernel methods stand out as a highly suitable and powerful approach.

Despite the superior performance of GPR-based hybrid models, this analysis confronted several notable challenges, primarily rooted in computational complexity and memory usage. Nonparametric methods such as kernel regression scale poorly with data size, kernel methods typically take on storage values of $\mathcal{O}(n^2)$ and computational complexity of $\mathcal{O}(n^3)$.²⁰ One of the main benefits of a hybrid kernel model revolves around the optimization of storage to $\mathcal{O}(n + m)$ and a computational complexity of $\mathcal{O}(n + m \log m)$ in part to the Toeplitz exploit.²¹ To ensure feasibility, the dataset was trimmed to include only the most recent decade (2015–2025), striking a balance between computational efficiency and relevance to modern market conditions. This choice ensured that models captured recent volatility regimes and structural changes while reducing processing time and storage overhead. Even with the truncated dataset, model training particularly for the hybrid method, was computationally intensive and required careful tuning. Additional hurdles included bandwidth selection, convergence in hyperparameter optimization, and managing numerical stability during matrix inversion, all of which underscore the trade-off between model expressiveness and practical tractability.

Looking ahead, several alternative approaches and extensions could enrich this analysis. With greater computational resources or longer timeframes, models could be trained on broader datasets, potentially enhancing generalization and the detection of long-term patterns. Structured approximations like inducing point methods or sparse variational Gaussian processes could be explored to reduce the computational burden without sacrificing model complexity. Beyond regression, classification techniques or neural network-based architectures may also capture

²⁰ Chen, D. (2023). A Hybrid Stochastic Interpolation and Compression Method for Kernel Matrices. *Journal of Computational Physics*, 494.

²¹ Ibid.

latent regimes in market volatility. Notably, hybrid kernel methods represent a growing frontier in machine learning research, combining the theoretical rigor of kernel methods with scalable computational strategies.²² As quantum computing matures, the matrix-heavy computations intrinsic to Gaussian processes may be accelerated, enabling real-time financial inference with complex kernel architectures. In the meantime, hybrid models remain a compelling option for high-stakes applications like volatility modeling, risk forecasting, and algorithmic trading, where accuracy and uncertainty quantification are paramount.

²² Zhang, J., Liu, C.-L., & Jiang, X. (2024). Polynomial Kernel Learning for Interpolation Kernel Machines with Application to Graph Classification. *Pattern Recognition Letters*, 186, 7–13.

Appendix

OLS Python Code:

Part I: Actual vs Predicted and Residual Plots

```
import pandas as pd
import statsmodels.api as sm
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

# Load the datasets

vix_df = pd.read_csv('/content/VIX_History.csv', index_col='DATE',
parse_dates=True)
vixy_df = pd.read_csv('/content/HistoricalIV_VIXY.csv', index_col='Date',
parse_dates=True)
# Select relevant columns and align the data based on the overlapping date
range
vix_data = vix_df[['CLOSE']].rename(columns={'CLOSE': 'VIX Close'})
vixy_iv = vixy_df[['IV30', 'OHLC 20-Day Vol']].dropna()

merged_df = pd.merge(vix_data, vixy_iv, left_index=True, right_index=True,
how='inner')

# Define the dependent and independent variables
dependent_var = merged_df['VIX Close']
independent_var = merged_df['IV30']

# OLS Model
# constant to the independent variable for the intercept
X = sm.add_constant(merged_df[independent_var.name])
y = merged_df[dependent_var.name]

ols_model = sm.OLS(y, X).fit()
ols_predictions = ols_model.predict(X) # Predict on the entire dataset

# Evaluate the OLS model
ols_mse = mean_squared_error(y, ols_predictions)
ols_rmse = np.sqrt(ols_mse)
ols_r2 = r2_score(y, ols_predictions)
```

```

print("\n Ordinary Least Squares (OLS) Model (Entire Dataset) ")
print(ols_model.summary())
print(f"Mean Squared Error (MSE): {ols_mse:.4f}")
print(f"Root Mean Squared Error (RMSE): {ols_rmse:.4f}")
print(f"R-squared (R^2): {ols_r2:.4f}")

# Plotting on the entire dataset
plt.figure(figsize=(12, 6))
plt.plot(merged_df.index, y, label='Actual VIX', color='blue')
plt.plot(merged_df.index, ols_predictions, label='OLS Predicted VIX',
color='red', linestyle='--')
plt.title('Actual VIX vs. OLS Predicted VIX (Entire Dataset)')
plt.xlabel('Date')
plt.ylabel('VIX Value')
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(12, 6))
residuals = y - ols_predictions
plt.plot(merged_df.index, residuals, label='OLS Residuals',
color='orange')
plt.title('OLS Residual Plot (Entire Dataset)')
plt.xlabel('Date')
plt.ylabel('Residuals (Actual - Predicted)')
plt.axhline(0, color='black', linestyle='--', linewidth=0.8)
plt.legend()
plt.grid(True)
plt.show()

```

Part II: Actual vs Predicted Volatility Plot

```

# Step 1: Install and import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import statsmodels.api as sm

```

```

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score, make_scorer
from sklearn.model_selection import KFold, cross_val_score
from statsmodels.nonparametric.kernel_regression import KernelReg

# Load the data
vix = pd.read_csv('/content/drive/MyDrive/vix.csv', parse_dates=['DATE'])
iv = pd.read_csv('/content/drive/MyDrive/HistoricalIV_VIXY.csv',
parse_dates=['Date'])

# Merge datasets
iv.rename(columns={'Date': 'DATE'}, inplace=True)
data = pd.merge(vix, iv, on='DATE', how='inner')

# Prepare the data for regression
X = data[['CLOSE']] # Predictor
y = data['IV30']     # Response

# OLS with statsmodels
X_sm = sm.add_constant(X)
ols_model = sm.OLS(y, X_sm).fit()

# Summary
print(ols_model.summary())

# Predictions
y_pred = ols_model.predict(X_sm)

# Evaluation Metrics
mse = mean_squared_error(y, y_pred)
r2 = r2_score(y, y_pred)
print(f"\nMean Squared Error: {mse:.4f}")
print(f"R-squared: {r2:.4f}")

# Cross-validation for OLS
kf = KFold(n_splits=5, shuffle=True, random_state=42)
ols_lr = LinearRegression()
ols_cv_scores = cross_val_score(ols_lr, X, y, cv=kf,
scoring='neg_mean_squared_error')
ols_cv_mse = -ols_cv_scores # Convert to positive MSE

```



```

print(f"\nOLS CV MSEs: {ols_cv_mse}")
print(f"Mean OLS CV MSE: {np.mean(ols_cv_mse):.4f}")

# Cross-validation for Kernel Regression
def kernel_cv_score(X, y, cv, var_type='c'):
    scores = []
    X = X.values.flatten()
    y = y.values
    for train_idx, test_idx in cv.split(X):
        X_train, y_train = X[train_idx], y[train_idx]
        X_test, y_test = X[test_idx], y[test_idx]

        model = KernelReg(endog=y_train, exog=X_train, var_type=var_type)
        y_pred, _ = model.fit(X_test)
        mse = mean_squared_error(y_test, y_pred)
        scores.append(mse)
    return scores

kernel_cv_scores = kernel_cv_score(X, y, kf)
print(f"\nKernel Regression CV MSEs: {kernel_cv_scores}")
print(f"Mean Kernel Regression CV MSE: {np.mean(kernel_cv_scores):.4f}")

# Plotting Actual vs Predicted
plt.figure(figsize=(10, 6))
plt.scatter(data['CLOSE'], y, label='Actual IV30', alpha=0.6)
plt.plot(data['CLOSE'], y_pred, color='red', label='OLS Predicted IV30')
plt.xlabel('VIX Close Price')
plt.ylabel('30-Day Implied Volatility (IV30)')
plt.title('OLS Regression: VIX Close vs IV30')
plt.legend()
plt.grid(True)
plt.show()

# Residual Plot
residuals = y - y_pred
plt.figure(figsize=(10, 6))
plt.scatter(y_pred, residuals, alpha=0.6)
plt.axhline(0, color='red', linestyle='--')
plt.xlabel('Predicted IV30')

```

```
plt.ylabel('Residuals')  
plt.title('Residual Plot for OLS Model')  
plt.grid(True)  
plt.show()
```

Kernel Regression Python Code:

Part I: Actual vs Predicted and Residual Plots

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler

# Load the datasets
try:
    vix_df = pd.read_csv('/content/drive/MyDrive/vix.csv',
index_col='DATE', parse_dates=True)
    vixy_df = pd.read_csv('/content/drive/MyDrive/HistoricalIV_VIXY.csv',
index_col='Date', parse_dates=True)
except FileNotFoundError:
    print("Error: One or both of the CSV files were not found.")
    exit()

# Select and align data
vix_data = vix_df[['CLOSE']].rename(columns={'CLOSE': 'VIX Close'})
vixy_iv = vixy_df[['IV30']].dropna()

merged_df = pd.merge(vix_data, vixy_iv, left_index=True, right_index=True,
how='inner')

# Filter data from 2015-01-01 onward
merged_df = merged_df[merged_df.index >= '2015-01-01']

# Train/Test Split based on date
train_df = merged_df[merged_df.index < '2020-01-01'] # Training set:
2015-2020
test_df = merged_df[merged_df.index >= '2015-01-01'] # Test set:
2015-2025

X_train = train_df[['IV30']].values
X_test = test_df[['IV30']].values
y_train = train_df['VIX Close'].values
y_test = test_df['VIX Close'].values

# Feature Scaling
```

```

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Manual Gaussian Kernel Regression
def gaussian_kernel(x1, x2, bandwidth):
    return np.exp(-np.sum((x1 - x2)**2, axis=1) / (2 * bandwidth**2))

def manual_kernel_regression(X_train, y_train, X_test, bandwidth):
    predictions = []
    for x_star in X_test:
        weights = gaussian_kernel(X_train, x_star, bandwidth)
        numerator = np.sum(weights * y_train)
        denominator = np.sum(weights)
        if denominator > 0:
            predictions.append(numerator / denominator)
        else:
            predictions.append(np.nan)
    return np.array(predictions)

# Choose a bandwidth
bandwidth = 1.0

# Make predictions (only 2023-2025 test period)
kernel_predictions = manual_kernel_regression(X_train_scaled, y_train,
X_test_scaled, bandwidth)

# Create full prediction series (NaN for training period)
full_kernel_predictions = pd.Series(index=merged_df.index, dtype=float)
full_kernel_predictions.loc[test_df.index] = kernel_predictions

# Evaluate only on the test set
kernel_mse = mean_squared_error(y_test, kernel_predictions)
kernel_rmse = np.sqrt(kernel_mse)
kernel_r2 = r2_score(y_test, kernel_predictions)

print("\n Manual Gaussian Kernel Regression ")
print(f"Bandwidth: {bandwidth}")
print(f"Mean Squared Error (MSE): {kernel_mse:.4f}")
print(f"Root Mean Squared Error (RMSE): {kernel_rmse:.4f}")

```

```
print(f"R-squared (R^2): {kernel_r2:.4f}")

# Plotting
plt.figure(figsize=(12, 6))
plt.plot(merged_df.index, merged_df['VIX Close'], label='Actual VIX',
color='blue')
plt.plot(full_kernel_predictions.index, full_kernel_predictions,
label='Kernel Predicted VIX', color='red', linestyle='--')
plt.title('Actual VIX vs. Kernel Predicted VIX (2015-2025)')
plt.xlabel('Date')
plt.ylabel('VIX Value')
plt.legend()
plt.grid(True)
plt.show()
```

Part II: Actual vs Predicted Volatility Plot

```
# Kernel Regression (Manual Gaussian Kernel) on VIX Close vs IV30

# Install and import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error, r2_score

# Load the data
data_vix = pd.read_csv('/content/drive/MyDrive/vix.csv')
data_iv = pd.read_csv('/content/drive/MyDrive/HistoricalIV_VIXY.csv')

# Convert dates
data_vix['DATE'] = pd.to_datetime(data_vix['DATE'])
data_iv['Date'] = pd.to_datetime(data_iv['Date'])

# Merge datasets on date
data = pd.merge(data_vix, data_iv, left_on='DATE', right_on='Date',
how='inner')

# Set features and target
X = data[['CLOSE']].values.flatten() # Feature: VIX closing price (1D
array)
y = data['IV30'].values # Target: 30-day implied volatility

# Define Gaussian Kernel function
def gaussian_kernel(x, xi, bandwidth):
    return np.exp(-0.5 * ((x - xi) / bandwidth) ** 2) / (bandwidth *
np.sqrt(2 * np.pi))

# Define Kernel Regression model
def kernel_regression(X_train, y_train, X_pred, bandwidth):
    y_pred = []
    for x0 in X_pred:
        weights = gaussian_kernel(x0, X_train, bandwidth)
        prediction = np.sum(weights * y_train) / np.sum(weights)
        y_pred.append(prediction)
    return np.array(y_pred)
```

```

# Set bandwidth manually
bandwidth = 1.0

# Run Kernel Regression
X_train = X
y_train = y
X_pred = X # Predicting on same X

y_kernel_pred = kernel_regression(X_train, y_train, X_pred, bandwidth)

# Evaluation Metrics
mse_kernel = mean_squared_error(y_train, y_kernel_pred)
rmse_kernel = np.sqrt(mse_kernel)
r2_kernel = r2_score(y_train, y_kernel_pred)

print(f"Kernel Regression Results:")
print(f"Bandwidth: {bandwidth}")
print(f"Mean Squared Error (MSE): {mse_kernel:.4f}")
print(f"Root Mean Squared Error (RMSE): {rmse_kernel:.4f}")
print(f"R-squared (R²): {r2_kernel:.4f}")

# Plot - Actual vs Predicted
plt.figure(figsize=(10, 6))
plt.scatter(X, y, label='Actual IV30', alpha=0.5)
plt.scatter(X, y_kernel_pred, color='red', label='Kernel Predicted IV30',
s=10)
plt.xlabel('VIX Close Price')
plt.ylabel('30-Day Implied Volatility (IV30)')
plt.title('Kernel Regression: VIX Close vs IV30')
plt.legend()
plt.grid(True)
plt.show()

# Plot - Residuals
residuals_kernel = y_train - y_kernel_pred
plt.figure(figsize=(10, 6))
plt.scatter(y_kernel_pred, residuals_kernel, alpha=0.5)
plt.axhline(0, color='red', linestyle='--')
plt.xlabel('Predicted IV30')
plt.ylabel('Residuals')

```

```
plt.title('Residual Plot for Kernel Regression')  
plt.grid(True)  
plt.show()
```


Hybrid Kernel Regression Python Code:

Part I: Actual vs Predicted and Residual Plots

```
# Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, ConstantKernel as C
from sklearn.metrics import mean_squared_error, r2_score

# Load Data
vix = pd.read_csv('/content/drive/MyDrive/vix.csv', parse_dates=['DATE'],
                  date_parser=lambda x: pd.to_datetime(x,
                  format='%m/%d/%Y'))
vix = vix.rename(columns={'DATE': 'Date'})
iv = pd.read_csv('/content/drive/MyDrive/HistoricalIV_VIXY.csv',
                  parse_dates=['Date'],
                  date_parser=lambda x: pd.to_datetime(x,
                  format='%m/%d/%Y'))

# Merge datasets on Date
merged_data = pd.merge(vix, iv, on='Date')

# Set 'Date' as index
merged_data = merged_data.set_index('Date')

# Filter the dataset to the last 10 years (from 2015-01-01 onward)
merged_data = merged_data[merged_data.index >= '2015-01-01']

# Step 3: Prepare Features (X) and Target (y)
X = np.arange(len(merged_data)).reshape(-1, 1) # Time index as feature
y = merged_data['CLOSE'].values                # Target: VIX closing
prices

# Step 4: Define Hybrid Model (KISS-GP)
# Using a simple hybrid kernel: Constant * RBF kernel
kernel = C(1.0, (1e-3, 1e3)) * RBF(length_scale=30.0,
length_scale_bounds=(1e-2, 1e3))
gpr = GaussianProcessRegressor(kernel=kernel, n_restarts_optimizer=5,
alpha=1e-2, normalize_y=True)
```

```

# Fit the Model
gpr.fit(X, y)

# Make Predictions
y_pred, y_std = gpr.predict(X, return_std=True)

# Evaluate Model
mse = mean_squared_error(y, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y, y_pred)

print(f"Hybrid Kernel Regression Results:")
print(f"Mean Squared Error (MSE): {mse:.4f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.4f}")
print(f"R-squared (R²): {r2:.4f}")

# Plot Actual vs Predicted VIX
plt.figure(figsize=(12, 6))
plt.plot(merged_data.index, y, label='Actual VIX', color='blue')
plt.plot(merged_data.index, y_pred, label='Predicted VIX (Hybrid Kernel)',
color='red')
plt.fill_between(merged_data.index, y_pred - 2*y_std, y_pred + 2*y_std,
color='pink', alpha=0.3)
plt.xlabel('Date')
plt.ylabel('VIX Closing Price')
plt.title('Actual vs Predicted VIX - Hybrid Kernel Regression')
plt.legend()
plt.grid(True)
plt.show()

# Plot Residuals
residuals = y - y_pred

plt.figure(figsize=(10, 5))
plt.scatter(y_pred, residuals, alpha=0.6)
plt.axhline(0, color='red', linestyle='--')
plt.xlabel('Predicted VIX')
plt.ylabel('Residuals')
plt.title('Residual Plot - Hybrid Kernel Regression')

```

```
plt.grid(True)  
plt.show()
```

Part II: Actual vs Predicted Volatility Plot

```
# Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, ConstantKernel as C
from sklearn.model_selection import train_test_split

# Load the data
vix_data = pd.read_csv('/content/drive/MyDrive/vix.csv')
iv_data = pd.read_csv('/content/drive/MyDrive/HistoricalIV_VIXY.csv')

# Format dates correctly
vix_data['DATE'] = pd.to_datetime(vix_data['DATE'], format='%m/%d/%Y')
iv_data['Date'] = pd.to_datetime(iv_data['Date'], format='%m/%d/%Y')

# Merge datasets on closest previous date
data = pd.merge_asof(iv_data.sort_values('Date'),
                     vix_data.sort_values('DATE'),
                     left_on='Date', right_on='DATE')

# Trim to last 10 years
cutoff_date = pd.to_datetime('2015-01-01')
data = data[data['Date'] >= cutoff_date]

# Prepare features (X) and target (y)
X = data[['IV30']].values
y = data['CLOSE'].values

# Train-test split (optional but good for evaluation)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

# Define and train the Hybrid Kernel (Gaussian Process) model
kernel = C(1.0) * RBF(length_scale=1.0)
gpr = GaussianProcessRegressor(kernel=kernel, n_restarts_optimizer=10,
                              alpha=1e-2, normalize_y=True)
gpr.fit(X_train, y_train)
```

```

# Predictions
y_pred, std_pred = gpr.predict(X, return_std=True)

# Plot - Actual vs Predicted (matching style)
plt.figure(figsize=(10, 6))
plt.scatter(y, X, label='Actual IV30', alpha=0.5)
plt.scatter(y_pred, X, color='red', label='Hybrid Predicted IV30', s=10)
plt.xlabel('VIX Close Price')
plt.ylabel('30-Day Implied Volatility (IV30)')
plt.title('Hybrid Kernel Regression: VIX Close vs IV30')
plt.legend()
plt.grid(True)
plt.show()

# Evaluate model performance
mse = mean_squared_error(y, y_pred)
r2 = r2_score(y, y_pred)

print(f"Hybrid Kernel MSE: {mse:.4f}")
print(f"Hybrid Kernel R2: {r2:.4f}")

from sklearn.model_selection import cross_val_score
from sklearn.metrics import make_scorer, mean_squared_error

# Use negative MSE because scikit-learn expects higher scores to be better
mse_scorer = make_scorer(mean_squared_error, greater_is_better=False)

# Compute 5-fold cross-validated MSE (take absolute value to interpret)
cv_mse_scores = cross_val_score(gpr, X, y, cv=5, scoring=mse_scorer)
mean_cv_mse = -np.mean(cv_mse_scores)

print("Cross-Validated MSE:", mean_cv_mse)

```

Data Set Sample

Historical VIX Data Obtained from CBOE:

VIX_History

DATE	OPEN	HIGH	LOW	CLOSE
01/02/1990	17.240000	17.240000	17.240000	17.240000
01/03/1990	18.190000	18.190000	18.190000	18.190000
01/04/1990	19.220000	19.220000	19.220000	19.220000
01/05/1990	20.110000	20.110000	20.110000	20.110000
01/08/1990	20.260000	20.260000	20.260000	20.260000
01/09/1990	22.200000	22.200000	22.200000	22.200000
01/10/1990	22.440000	22.440000	22.440000	22.440000
01/11/1990	20.050000	20.050000	20.050000	20.050000
01/12/1990	24.640000	24.640000	24.640000	24.640000
01/15/1990	26.340000	26.340000	26.340000	26.340000
01/16/1990	24.180000	24.180000	24.180000	24.180000
01/17/1990	24.160000	24.160000	24.160000	24.160000
01/18/1990	24.340000	24.340000	24.340000	24.340000
01/19/1990	22.500000	22.500000	22.500000	22.500000
01/22/1990	26.700000	26.700000	26.700000	26.700000
01/23/1990	24.720000	24.720000	24.720000	24.720000

Historical Implied Volatility Obtained from Market Chameleon:

HistoricalIV_VIXY

Date	IV30	IV30 20-Day MA	OHLC 20-Day Vol	OHLC 52-Week Vol	Options Volume
4/17/2025	92.34	100.852	137.4726	81.1291	2497
4/16/2025	115.8	99.4505	136.5153	81.4687	2075
4/15/2025	92.33	96.928	136.4212	81.7168	1741
4/14/2025	101.77	95.889	131.7685	81.0323	1345
4/11/2025	127.9	94.059	131.5032	80.7563	1675
4/10/2025	149.52	91.152	122.2853	79.2384	1941
4/9/2025	118.21	88.3695	111.3769	77.0515	2972
4/8/2025	190.09	86.9015	102.3398	75.2907	2709
4/7/2025	163.24	82.9615	95.2254	73.8855	3738
4/4/2025	163.69	80.33	86.4328	72.57	6372
4/3/2025	111.63	76.475	80.5252	71.1383	4811
4/2/2025	71.52	76.3855	77.862	70.728	1634
4/1/2025	73.04	77.167	81.7619	70.5856	1057
3/31/2025	70.86	77.832	84.4521	70.2079	2103
3/28/2025	73.23	78.8955	81.3121	69.7638	1893
3/27/2025	61.88	78.606	84.9952	69.7372	753

Bibliography

- Ait-Sahalia, Y., Bickel, P. J., & Stoker, T. M. (n.d.). Goodness-of-fit Tests for Kernel Regression with an Application to Option Implied Volatilities. *Journal of Econometrics*, 105(2), 363–412.
- Cboe Global Markets. (n.d.). *Cboe Volatility Index® (VIX®)*. Retrieved April 19, 2025, from https://www.cboe.com/tradable_products/vix/
- Chen, D. (2023). A Hybrid Stochastic Interpolation and Compression Method for Kernel Matrices. *Journal of Computational Physics*, 494.
- Dalderop, J., & Linton, O. B. (2024). Estimating a Density Ratio Model for Stock Market Risk and Option Demand. *Cambridge Working Papers in Economics*, 2411.
- Market Chameleon. (n.d.). *VIXY - ProShares VIX Short-Term Futures ETF Options Chain & Implied Volatility*. Retrieved April 19, 2025, from <https://marketchameleon.com/Overview/VIXY/OptionChain>
- Nickolas, S. (2022, May 20). *Implied volatility: What it means and how it works*. Investopedia. <https://www.investopedia.com/terms/i/impliedvolatility.asp>
- Racine, J. S. (2008b). Nonparametric Econometrics: A Primer. *Foundations and Trends in Econometrics*, 3(1).
- Sadek, A., & Mohammed, L. A. (2024). Evaluation of the Performance of Kernel Nonparametric Regression and Ordinary Least Squares Regression. *International Journal on Informatics Visualization*, 8(3).
- Wilson, A. G., & Nickisch, H. (2015). *Kernel Interpolation for Scalable Structured Gaussian Processes (KISS-GP)*. <https://doi.org/https://doi.org/10.48550/arXiv.1503.01007>
- Yatchew, A. (1998). Nonparametric Regression Techniques in Economics. *Journal of Economic Literature*, 36(2), 669–721.
- Zhang, J., Liu, C.-L., & Jiang, X. (2024). Polynomial Kernel Learning for Interpolation Kernel Machines with Application to Graph Classification. *Pattern Recognition Letters*, 186, 7–13. <https://doi.org/https://doi.org/10.1016/j.patrec.2024.08.022>